ASE CHAPTER 7

T5 Applied Scenarios & Interpretation Pack

Scenario 7.1 · Operational Profile Construction & Test Case Allocation

An enterprise software team is developing a high-throughput automated banking system handling four core customer transactions: *Cash Withdrawal*, *Fund Transfer*, *Balance Inquiry*, and *Bill Payment*. Historical production data indicates that out of 500,000 daily transactions, 250,000 are Cash Withdrawals, 125,000 are Balance Inquiries, 75,000 are Fund Transfers, and 50,000 are Bill Payments.

1. Construct the quantitative operational profile for this banking application.
2. If the quality assurance budget permits running a suite of 20,000 automated operational test cases before deployment, calculate how test cases should be distributed across the four transaction types.
3. Explain how testing in accordance with this operational profile may affect reliability growth compared with equal test distribution.

1. Quantitative Operational Profile Construction:

- Total daily transactions $N = 500,000$.
- Probability of Cash Withdrawal:

$$P(\text{Withdrawal}) = \frac{250,000}{500,000} = 0.50 \quad (50\%)$$

- Probability of Balance Inquiry:

$$P(\text{Inquiry}) = \frac{125,000}{500,000} = 0.25 \quad (25\%)$$

- Probability of Fund Transfer:

$$P(\text{Transfer}) = \frac{75,000}{500,000} = 0.15 \quad (15\%)$$

- Probability of Bill Payment:

$$P(\text{Payment}) = \frac{50,000}{500,000} = 0.10 \quad (10\%)$$

- Sum of probabilities: $0.50 + 0.25 + 0.15 + 0.10 = 1.00$.

2. Distribution of 20,000 Automated Test Cases:

- **Cash Withdrawal:** $20,000 \times 0.50 = 10,000$ test cases
- **Balance Inquiry:** $20,000 \times 0.25 = 5,000$ test cases
- **Fund Transfer:** $20,000 \times 0.15 = 3,000$ test cases
- **Bill Payment:** $20,000 \times 0.10 = 2,000$ test cases

3. Reliability Growth Acceleration Rationale:

- **Faults residing in frequently executed code paths have a vastly higher probability of activation during live operation.**
- If tests are distributed uniformly ($5,000$ tests per transaction), Cash Withdrawal (which represents 50% of observed transactions) would be severely undertested, leaving high-exposure defects dormant until release.

- Allocating 50% of the test cases to Cash Withdrawal aligns the test distribution with observed use. This can expose faults in frequently used transaction paths earlier, but it does not guarantee the fastest possible reliability growth.

> **Academic Note**
>
> **القاعدة الهندسية:** توزيع الاختبار وفق البروفايل التشغيلي يطابق كثافة الاختبارات مع الاستخدام المرصود ← وقد يساعد على كشف أخطاء المسارات الشائعة مبكراً، لكنه لا يضمن أسرع نمو ممكن للموثوقية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.5, pp. 316–318; companion slides 16–20.

Scenario 7.2 · Objective-Driven Software Release Certification

A flight-navigation software package is undergoing system testing. The project contract specifies a mandatory release reliability requirement: **Failure intensity must not exceed** $\lambda_F = 0.20\ \text{failure/CPU hr}$. The software development records an initial failure intensity $\lambda_0 = 8.0\ \text{failures/CPU hr}$ and total expected failures $v_0 = 160\ \text{failures}$ under the Basic Execution Time Model. Testing has currently executed for 40 CPU hours and experienced 110 failures.

1. Determine the current failure intensity λ_P and evaluate whether the system currently satisfies the contractual release criterion.
2. Calculate the additional number of failures that must be discovered and resolved, and the additional CPU execution hours required to achieve contractual release readiness.
3. What engineering risks arise if management forces early deployment based on calendar deadline rather than the quantitative failure intensity objective?

1. Current Failure Intensity Evaluation:

- Current failures experienced: $\mu_P = 110\ \text{failures}$.
- Using the Basic Execution Time Model:

$$\lambda_P = \lambda_0 \left(1 - \frac{\mu_P}{v_0}\right) = 8.0 \times \left(1 - \frac{110}{160}\right) = 8.0 \times (1 - 0.6875) = 8.0 \times 0.3125 = 2.50\ \text{failures/CPU hr}$$

- **Release Certification Decision: The software CANNOT be released.** Current failure intensity ($\lambda_P = 2.50\ \text{failures/CPU hr}$) is more than **12 times higher** than the contractual safety threshold ($\lambda_F = 0.20\ \text{failure/CPU hr}$).

2. Additional Testing Effort Required:

- **Additional failures to experience ($\Delta\mu$):**

$$\Delta\mu = \frac{v_0}{\lambda_0}(\lambda_P - \lambda_F) = \frac{160}{8.0}(2.50 - 0.20) = 20 \times 2.30 = 46\ \text{failures}$$

  (Total failures at release: $\mu_F = 160(1 - 0.20/8.0) = 160 \times 0.975 = 156$ failures; $\Delta\mu = 156 - 110 = 46$).

- **Additional execution time required ($\Delta\tau$):**

$$\Delta\tau = \frac{v_0}{\lambda_0} \ln\left(\frac{\lambda_P}{\lambda_F}\right) = \frac{160}{8.0} \ln\left(\frac{2.50}{0.20}\right) = 20 \times \ln(12.5) = 20 \times 2.525729 \approx 50.51\ \text{CPU hrs}$$

3. Engineering Risks of Premature Release:

- The computed failure intensity is above the contract's stated release limit, so the software does not meet the given release criterion. The scenario does not establish the physical consequences of a failure or any post-deployment cost multiplier.

📝 **Academic Note**

قرار الإطلاق: البرنامج لا يحقق شرط الإطلاق المعطى لأن شدة الفشل المحسوبة أعلى من الحد التعاقدي ← يلزم وفق الحساب اكتشاف 46 فشلاً إضافياً وتنفيذ نحو 50.51 ساعة معالج إضافية.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.8 & §7.3.1, pp. 320–322, 329–334; companion slides 21–23, 47–58.

Scenario 7.3 · Environmental Shift & Input Space Activation

A commercial logistics inventory software product was tested rigorously in the software vendor's laboratory for 6 months, exhibiting zero failures across 500 consecutive test hours ($MTTF > 500$ hrs). However, within 48 hours of installation at a customer warehouse in a foreign country, the system crashed multiple times per day.

1. Using the concepts of **Input Space (I)**, **Failure Domain (I_F)**, and **Operational Profile**, explain how software can be failure-free in the laboratory yet fail immediately in the field.
2. Identify the technical flaws in the vendor's testing strategy that permitted this discrepancy.
3. Formulate three concrete corrective actions to prevent environmental failure discrepancies in future releases.

1. Possible Explanation Using Input Space and Operational Profile:

- A latent fault may be activated only by some inputs in the system's input space I, forming a failure domain $I_F \subset I$.
- The zero observed failures in 500 laboratory hours do not prove that the laboratory inputs were disjoint from I_F, nor do they establish 100% reliability. They show only that no failure was observed during the stated test period.
- A difference between laboratory and field operational profiles may have exposed inputs that activate a fault. Locale-specific text, date or currency formats, concurrency, and operating-system settings are possible investigation areas; the scenario provides no evidence that any one of them caused these failures.

2. Flaws in the Vendor's Testing Strategy:

- The scenario does not describe the test inputs or environment, so a specific testing flaw cannot be confirmed. A likely gap to investigate is whether the tests represented the customer's operational profile and environment.
- Review the laboratory test data, field logs, configurations, and input distributions to determine which conditions differ and whether those conditions activate a latent fault.

3. Corrective Engineering Actions:

- **Field Telemetry and Log Analysis:** Capture real customer transaction traces and environmental parameters to build representative operational profiles.
- **Beta Testing and Pilot Field Deployments:** Deploy release candidates in customer staging environments before commercial release.
- **Robust Input Boundary Validation:** Implement defensive programming at all external input interfaces to reject or sanitize inputs outside specified tolerances before they propagate into business logic.

> 📝 **Academic Note**
> **التعليل الهندسي:** قد يفعّل مدخل مغطى بمنطقة الفشل (I_F) عيباً كامناً ← عدم ظهور فشل خلال الاختبار المعملي لا يثبت خلو النظام من الأخطاء، واختلاف بيئة العميل فرضية تحتاج إلى فحص السجلات والبيانات قبل اعتباره سبباً مؤكداً.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.4 & §7.1.5, pp. 314–318; companion slides 16–20.

Scenario 7.4 · SEI CMM Level 2 Readiness Appraisal & Gap Remediation

An independent appraisal assesses a software development organization aiming for SEI CMM Level 2 (Repeatable). The findings reveal the following current organizational practices:

- Project budgets and schedules are estimated informally by project managers based on intuition without historical tracking data.
- System requirements change constantly via verbal phone calls and emails from clients without formal documentation or impact assessments.
- Code source files are stored on developer local hard drives and merged manually via shared network folders without a formal version control tool.
- A dedicated Software Quality Assurance (SQA) group conducts testing at the end of the project, but has no authority to review project plans or audit adherence to development standards.

- Map these audit findings to the 6 Key Process Areas (KPAs) of CMM Level 2 and determine whether the organization qualifies for Level 2 maturity.

- Detail the specific institutional deficiencies across the failed KPAs.
- Formulate an actionable remediation plan outlining practices that would help address the observed gaps before a Level 2 appraisal.

1. KPA Mapping & Appraisal Determination:

- Informal schedule and budget estimation violates **Software Project Planning (SPP)** and **Software Project Tracking and Oversight (SPTO)**.
- Uncontrolled requirements changes violate **Requirements Management (RM)**.
- Manual file merging without tool-enforced versioning violates **Software Configuration Management (SCM)**.
- SQA lacking audit authority over processes and plans violates **Software Quality Assurance (SQA)**.
- **Appraisal conclusion:** The documented gaps are inconsistent with Level 2 practices, so the organization is not ready to demonstrate Level 2 based on this evidence. The scenario gives no information about Software Subcontract Management and is not a complete appraisal; it does not establish the organization's precise current maturity level.

2. Specific Institutional Deficiencies:

- **Absence of Baseline Control (SCM):** High risk of code regression, accidental overwrites, and unverified software releases.
- **Requirements Management (RM):** Uncontrolled requirement changes increase the risk of scope, schedule, and cost deviations.
- **Subjective Project Management (SPP/SPTO):** Lack of disciplined estimation models (e.g., COCOMO, Function Points) or tracking against documented project plans.
- **Weak Quality Governance (SQA):** SQA functions merely as late-stage testing rather than an independent process-compliance auditor reporting directly to senior executive management.

3. Actionable Remediation Plan:

- **Institutionalize Requirements Management:** Establish a formal Change Control Board (CCB) and documented baseline procedure; all requirements modifications must undergo formal impact analysis and customer sign-off before adoption.
- **Establish Formal Project Planning & Tracking:** Adopt standard estimation tools and create documented Software Development Plans (SDPs); conduct bi-weekly formal reviews comparing actual effort, cost, and milestones against the plan.
- **Implement Configuration Management:** Deploy a centralized, secure version control system; establish baseline control for all source code, design documents, and test artifacts.
- **Empower SQA:** Mandate that SQA conduct periodic audits of engineering activities and work products against documented standards, with direct reporting lines to executive management independent of project managers.

📝 **Academic Note**

شروط بلوغ المستوى الثاني (Repeatable): المعطيات تثبت وجود فجوات في مجالات المستوى الثاني، لكنها لا تكفي لتحديد مستوى المؤسسة الحالي أو الحكم على SSM ← يجب استكمال التقييم ومعالجة الفجوات المثبتة قبل تقرير الجاهزية للمستوى الثاني.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.1 & §7.4.2, pp. 347–353; companion slides 94–101.

Scenario 7.5 · Post-Release Maintenance Failure Spikes & Regression Analysis

A cloud enterprise software system achieved its target failure intensity of $\lambda = 0.10\ \text{failure}/\text{CPU hr}$ and was successfully deployed into production. Two months later, the development team released "Service Pack 1" to resolve 15 minor user interface complaints and add a small billing export feature. Within three days of deploying Service Pack 1, production failure intensity suddenly surged to $\lambda = 1.85\ \text{failures}/\text{CPU hr}$, triggering emergency system restarts.

1. Explain this phenomenon using the **Software Reliability Curve** (Figure 7.2) and contrast it with hardware maintenance behavior.
2. Identify the primary engineering cause of this reliability degradation.
3. Formulate a rigorous software maintenance and regression testing protocol to ensure that future maintenance patches do not compromise operational reliability baselines.

1. Reliability Curve Analysis:

- In hardware, maintenance replaces physically worn components with identical spares, reliably restoring the failure rate to its low useful-life baseline.
- In software, **maintenance does not replace physical parts; it alters the logical source code.**
- **Software modifications can introduce regression faults, causing sudden upward spikes in failure intensity.** Here, the increase from 0.10 to $1.85\ \text{failures}/\text{CPU hr}$ illustrates a maintenance-induced failure spike.

2. Plausible Engineering Cause:

- A regression fault introduced by the maintenance change is a plausible explanation for the increase. The timing alone does not identify the faulty change or prove that shared variables, database schemas, or API contracts were involved.
- The scenario gives no details about pre-release testing. Check the change set, failure logs, and test results to determine whether regression testing missed the affected behavior.

3. Remediation & Regression Protocol:

- **Automated Continuous Regression Testing:** Prior to releasing any maintenance patch or hotfix, the entire baseline regression test suite must execute automatically to verify that existing functionality remains intact.
- **Formal Peer Reviews of Changes:** Mandate formal code inspections for all maintenance edits, evaluating not only the fix itself but also potential side effects on dependent modules.
- **Configuration Management Baselines:** Maintain strict branch tagging; if post-release telemetry detects an abnormal failure intensity spike, implement automated rollback to the previously certified stable baseline.

📝 **Academic Note**

تفسير قفزة الفشل (Failure Spike): صيانة البرمجيات تغير المنطق البرمجي وليست مجرد استبدال قطع ← أي تعديل طفيف قد يولد أخطاء راجعة (Regression faults) في أجزاء كانت سليمة، ما لم يتم إجراء اختبار انحدار شامل (Regression testing) قبل الإطلاق.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.1.1, pp. 308–310; companion slides 3–5.

Scenario 7.6 · Standards Selection Strategy: ISO 9001 vs SEI CMM Appraisal

A mid-sized software engineering firm (120 developers) currently holds no formal quality certifications. The board of directors debates whether to invest corporate budget into pursuing **ISO 9001 certification** or **SEI CMM Level 3 appraisal**. The firm specializes in developing safety-critical medical device software for hospital networks and military defense contractors.

1. Analyze the strategic suitability, strengths, and limitations of ISO 9001 versus SEI CMM for this specific company.
2. Provide a justified recommendation on which quality framework the organization should prioritize.
3. If the company ultimately decides to pursue both standards, explain the optimal chronological sequence of implementation and how progress in one framework accelerates compliance with the other.

1. Strategic comparison:

- **ISO 9001:** A general quality-management standard. It may matter if a customer or market requires certification, but its broad requirements need interpretation for software work.
- **CMM:** Focuses on software processes and gives more detailed guidance for continuous process improvement. It can be applied to development and maintenance, though it does not directly cover every maintenance-specific concern.

2. Recommendation under a stated assumption: If the board's main objective is to improve the firm's software processes and no current contract requires ISO certification, prioritize CMM-based improvement. If a customer, tender, or applicable requirement explicitly requires ISO 9001 certification, include that requirement in the plan; the scenario does not provide enough information to rank certification above the process-improvement objective.

3. If pursuing both:

- First, identify any customer or market certification requirements and the target audit scope.
- Use CMM to guide software-process improvement where that is the chosen objective.
- Conduct a separate ISO 9001 gap assessment. CMM practices may support parts of the quality system, but the relationship is not one-to-one and a CMM maturity level does not automatically establish ISO 9001 conformity.
- Schedule certification only after the organization has verified and implemented the applicable ISO requirements.

Neither framework by itself should be treated as proof that a particular software product is safe or reliable.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), §7.4.3–§7.5.3, pp. 353–357; companion slides 103–107.
